

Software Requirements Specification

for

Athena : A Study Management Tool for Students

Version 1.0 approved

**Prepared by Ritika Dhole
Shreyas Nikam
Aaditya Ghorpade**

MIT WPU Pune

28-07-2026

Table of Contents

Table of Contents.....	ii
Revision History	ii
Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope	1
1.5 References.....	1
2. Overall Description.....	2
2.1 Product Perspective	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics.....	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints.....	2
2.6 User Documentation.....	2
2.7 Assumptions and Dependencies.....	3
3. External Interface Requirements.....	3
3.1 User Interfaces.....	3
3.2 Hardware Interfaces.....	3
3.3 Software Interfaces.....	3
3.4 Communications Interfaces.....	3
4. System Features	4
4.1 System Feature 1	4
4.2 System Feature 2 (and so on).....	4
5. Other Nonfunctional Requirements.....	4
5.1 Performance Requirements.....	4
5.2 Safety Requirements.....	5
5.3 Security Requirements.....	5
5.4 Software Quality Attributes.....	5
5.5 Business Rules.....	5
6. Other Requirements	5
Appendix A: Glossary.....	5
Appendix B: Analysis Models.....	5
Appendix C: To Be Determined List.....	6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for Athena, a web-based Study Management Tool, Version 1.0. Athena is designed to help students manage study materials, deadlines, schedules, and group collaboration through a single, unified platform, replacing the disconnected tools students currently rely on for these tasks.

This document covers the complete scope of Athena's initial release, including resource management, an integrated PDF viewer, calendar synchronization, to-do tracking, a study timer, and study group collaboration. It is intended to establish a shared understanding of Athena's objectives, functional scope, and requirements among everyone involved before design and development begin.

This SRS is meant to be used by developers, project supervisors, and evaluators as the primary reference for what Athena needs to do, and serves as the baseline against which the completed platform will be assessed.

1.2 Document Conventions

This document follows the IEEE 830 structure for Software Requirements Specifications, adapted to the needs of the Study Management Tool project. The conventions below are meant to keep every requirement clear, traceable, and easy to verify through design, development, and testing.

Requirement Identification

Every functional requirement carries a unique identifier so it can be traced across design, implementation, and testing. Identifiers follow a module-based pattern, for example:

- REQ-TODO-001 – to-do tracker requirements
- REQ-TIMER-004 – study timer requirements
- REQ-PDF-007 – PDF viewer requirements
- REQ-CAL-002 – calendar synchronization requirements
- REQ-COLLAB-010 – study group collaboration requirements

Once assigned, an identifier stays fixed for the life of the project, even if the requirement's wording changes in a later revision.

Requirement Priority

Each requirement carries a priority indicating how important it is to the overall product:

Priority	Description
Critical	Needed for Athena to function at all; without it, the release isn't usable.
High	Core functionality expected in the first working release of a module.
Medium	Improves usability or convenience but could slip to a later increment.
Low	A nice-to-have, likely a stretch goal if time runs short.

Unless stated otherwise, a detailed requirement takes on the priority of the feature it belongs to.

Terminology

A few terms are used consistently throughout this document:

- **Athena** refers to the Study Management Tool platform as a whole.
- **Student** refers to any authenticated person using Athena.
- **Increment** refers to one functional slice of the project (e.g., to-do tracker + study timer) delivered as a self-contained, working unit.

- **Module** refers to a distinct functional area of Athena, such as the PDF viewer or calendar sync.
- **Session** refers to a single continuous period of study timer usage (e.g., one Pomodoro cycle).

Diagrams and Models

Where helpful, this document includes visual models such as phase diagrams, ER diagrams, and wireframes. These are meant to support and clarify the written requirements, not replace them — they should be read alongside the corresponding text rather than on their own.

Document Relationships

This document serves as the primary Software Requirements Specification for Athena. It defines the overall scope, features, and requirements for the initial release. It does not attempt to cover implementation details such as database schema design, API contracts, or UI mockups in depth — those would sit in supporting design documents (e.g., ER diagrams, wireframes, architecture notes) referenced or attached alongside this SRS as the project progresses.

1.3 Intended Audience and Reading Suggestions

This document is meant for everyone involved in planning, building, and evaluating Athena.

- **Faculty Evaluators**

Interested in the project's scope, chosen development approach, and overall feasibility. Recommended reading: Introduction, Product Scope, Overall Description, System Features.

- **Developers (Project Team)**

Responsible for implementing the modules described in this document.

Recommended reading: Entire document, particularly System Features and External Interface Requirements.

- **Testers**

Responsible for verifying that each module behaves as specified.

Recommended reading: System Features, Functional Requirements, Nonfunctional Requirements.

- **Project Managers / Team Leads**

Responsible for tracking progress across increments and coordinating the team.

Recommended reading: Introduction, Overall Description, System Features (for scope and priority).

- **Future Contributors / Maintainers**

Anyone extending Athena after the initial release.

Recommended reading: Entire document, with particular attention to Design and Implementation Constraints and Assumptions and Dependencies.

This document is organized as follows:

1. **Introduction** : Includes purpose, scope, and conventions used throughout the SRS.
2. **Overall Description** : Provides a high-level look at what Athena does, who uses it, and the environment it runs in.
3. **External Interface Requirements** : Mentions how students can interact with Athena and how it connects to external services.
4. **System Features**: Provides a detailed, module-by-module breakdown of functional requirements (to-do tracker, PDF viewer, calendar sync, study timer, collaboration).
5. **Other Nonfunctional Requirements**: Specifies performance, security, and quality expectations.
6. **Other Requirements and Appendices**: Specifies supporting material, glossary, and models.

Readers new to the project are encouraged to start with the Introduction and Overall Description before moving into the module-specific detail in System Features, since the earlier sections set up context that makes the later requirements easier to follow.

1.4 Product Scope

Athena is a web-based Study Management Tool built to bring together the scattered pieces of a student's academic life into one place. Instead of switching between a separate to-do app, PDF reader, calendar, timer, and group chat, students would get all of these tools inside a single platform.

The initial release of Athena focuses on six core capabilities:

- **To-do tracker** — lets students create, organize, and track academic tasks and deadlines in one place.
- **Study timer** — offers a Pomodoro-style timer to help structure focused study sessions and build consistent study habits.
- **Resource manager and PDF viewer** — allows students to upload, organize, and annotate study material directly in the browser.
- **Calendar synchronization** — pulls deadlines and events from Google Calendar or Outlook, so students don't have to copy dates across apps manually.
- **Study group collaboration** — brings shared folders, group notes, and discussion into the platform for students working together.

The goal behind Athena is simple: reduce the friction of juggling multiple disconnected tools so students can spend less time managing their study process and more time actually studying. By centralizing resources, deadlines, and collaboration in one platform, Athena aims to improve organization, reduce missed deadlines, and make group study more coordinated.

This release establishes the foundational feature set for Athena. Future versions could expand on this with additional integrations, mobile support, or more advanced analytics on study habits, but those are outside the scope of this document.

1.5 References

The following references provide the foundation for this Software Requirements Specification Standards

- IEEE Std 830-1998 – *IEEE Recommended Practice for Software Requirements Specifications*
- IEEE 29148 – *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*
- ISO 9001:2015 – *Quality Management Systems*
- ISO/IEC 25010 – *Systems and Software Quality Model*
- BPMN 2.0 (Business Process Model and Notation)
- UML 2.x Specification (Object Management Group)

Technology References

- Google Calendar API Documentation
- Microsoft Outlook Calendar API Documentation
- PDF.js Documentation (for in-browser PDF rendering)
- React Documentation
- Node.js / Express Documentation

2. Overall Description

2.1 Product Perspective

Athena is a new, self-contained web application, not a follow-on to an existing product or a replacement for any system currently in use. It is being built from the ground up specifically to address the problem of students relying on multiple disconnected tools — separate apps for to-do lists, PDF readers, calendars, timers, and group chat — to manage their study routine.

Rather than replacing any one of these tools individually, Athena brings their core functionality together under one platform, so students don't need to switch between separate applications to manage different parts of their academic workflow.

Athena is built around a central web application shell that provides authentication and a

common user interface, with five functional modules plugging into it:

- To-do tracker
- Study timer
- Resource manager and PDF viewer
- Calendar synchronization
- Study group collaboration

Each module operates largely on its own but shares the same underlying student account, authentication, and base database, which lets them work together — for example, a deadline pulled in through calendar sync can show up directly in the to-do tracker.

Athena also connects to two external services: Google Calendar and Outlook, for pulling in deadlines and events. Beyond these two integrations, Athena is designed to work as a standalone platform and does not depend on any other larger system.

2.2 Product Functions

Athena needs to let students carry out the following major functions, organized by module. Detailed requirements for each of these appear in Section 4.

To-Do Tracker

- Create, edit, and delete academic tasks
- Set deadlines and priority levels for each task
- Mark tasks as complete and track overall progress

Study Timer

- Start, pause, and reset a Pomodoro-style study session
- Log completed study sessions against the student's account
- View a history of past study sessions

Resource Manager and PDF Viewer

- Upload and organize study material into folders
- View PDFs directly in-browser without a separate reader
- Annotate and highlight PDF content

Calendar Synchronization

- Connect a Google Calendar or Outlook account
- Pull in deadlines and events automatically
- Reflect synced events within the to-do tracker

Study Group Collaboration

- Create and join study groups
- Share folders and resources with group members
- Post and reply to group notes and discussion threads

Together, these functions let a student move from managing tasks and deadlines, to studying material, to coordinating with peers, all without leaving Athena. The diagram below shows how the modules connect to the shared core skeleton (authentication, database, base UI) described in Section 2.1.

2.3 User Classes and Characteristics

Athena is designed primarily for students, but different types of people would interact with the platform in different ways depending on how they study and organize their coursework.

User Class	Primary Use	Technical Expertise	Frequency of Use	Importance
Individual Student	Personal organization: tracking tasks, running study timer sessions, and managing personal study material through the PDF viewer	Low to Medium	Daily	Most important, represents the core individual productivity use case

Group Study Participant	Uses collaboration features alongside personal tools: joining study groups, sharing resources, participating in group notes and discussions	Low to Medium	Several times a week, tied to active group projects	Most important, represents the core collaborative use case
Deadline-Driven Student	Relies heavily on calendar synchronization and the to-do tracker to stay on top of coursework across subjects	Low	Daily, mainly around deadline periods	Moderate, narrower use pattern focused on one or two modules
Occasional/Light User	Uses Athena sporadically, often only during exam periods, mainly for the to-do tracker and PDF viewer	Low	Occasional, tied to exam or assignment cycles	Lower, needs are met by core features alone

Since Athena is aimed at a general student population rather than a technical or specialist audience, none of these classes require advanced technical skill to use the platform. The Individual Student and Group Study Participant are considered the two most important classes to satisfy for the initial release, since together they represent the two primary use patterns the platform is built around: personal productivity and collaborative study.]

2.4 Operating Environment

Athena operates as a web-based application accessible through standard modern browsers, so no dedicated hardware or installation is required on the student's end.

Client Environment

- Modern web browsers: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari
- Desktop, laptop, and tablet devices with a minimum screen width of 1024 pixels
- Stable internet connection needed for calendar sync, collaboration features, and file uploads

Server Environment

- Linux-based server (Ubuntu Server recommended)
- Node.js runtime for the backend application server
- PostgreSQL or MongoDB for the primary database
- Cloud storage (e.g., AWS S3 or equivalent) for hosting uploaded PDFs and study material

Network Requirements

- HTTPS for all client-server communication
- WebSocket support for real-time features in study group collaboration
- Outbound API access to Google Calendar and Microsoft Outlook for calendar synchronization

External Software Dependencies Athena needs to coexist with the following third-party services and libraries:

- Google Calendar API and Microsoft Outlook Calendar API for calendar sync
- PDF.js (or a comparable library) for in-browser PDF rendering and annotation
- An authentication provider (e.g., OAuth 2.0 via Google or a custom auth service) for account login

Since Athena is built as a lightweight academic tool rather than an enterprise platform, it doesn't require specialized server hardware beyond what's typical for a small to medium web application, and it's expected to run comfortably on a single cloud instance for the scope of this project.

2.5 Design and Implementation Constraints

A few constraints shape how Athena needs to be designed and built.

Athena is expected to:

- Use Node.js as the backend runtime and Express (or a comparable framework) for the API layer.
- Use PostgreSQL or MongoDB as the primary database for storing student accounts, tasks, and study session data.
- Use React for the frontend, keeping the interface consistent across all five modules.
- Store uploaded PDFs and study material in cloud storage rather than on the application server itself, to keep the platform lightweight and scalable.
- Rely on PDF.js (or an equivalent library) for in-browser PDF rendering and annotation, since building a custom PDF renderer is outside the scope of this project.
- Integrate with the Google Calendar API and Microsoft Outlook Calendar API for calendar synchronization, which means Athena's sync features depend on the availability and rate limits of these third-party services.
- Use HTTPS for all client-server communication to keep student data secure in transit.
- Support WebSocket connections for real-time updates in the study group collaboration module.
- Follow a modular architecture, with each of the five features (to-do tracker, timer, PDF viewer, calendar sync, collaboration) built as a distinct component that plugs into the shared authentication and database layer, matching the incremental development approach chosen for this project.

Since this is an academic project built within a single semester, development time and team size are also practical constraints. This favors the incremental approach already selected, since it lets core features be completed and demoed even if later increments (such as calendar sync or collaboration) face delays due to third-party API integration.

There are no specific corporate, regulatory, or organizational policies governing this project, since Athena is being developed as a semester project rather than for external commercial deployment.

2.5 User Documentation

The following documentation would accompany Athena's initial release, helping students get oriented and make full use of the platform.

- **Getting Started Guide** : a short walkthrough covering account setup, connecting a calendar, and an overview of the five core modules.
- **In-App Tooltips and Help Text** : contextual hints placed directly in the interface, especially for less obvious actions like PDF annotation and calendar sync setup.
- **FAQ Page** : answers to common questions, such as troubleshooting calendar sync issues or recovering a forgotten password.

Documentation would be delivered in Markdown or HTML format, accessible directly within the application or through a linked help page, so students don't need to leave the platform to find guidance. Since Athena is a semester project rather than a commercial product, formal documentation such as a full user manual or printed guide is not planned for the initial release; the focus stays on lightweight, in-context help that gets a student productive quickly.

2.7 Assumptions and Dependencies

The following assumptions were made while developing the requirements for Athena. If any of these turn out to be incorrect or change during the project, the affected requirements would need to be revisited.

Assumptions:

- Students accessing Athena have a stable internet connection, since core features like calendar sync, collaboration, and cloud-stored PDFs depend on connectivity.
- Students already hold an active Google or Microsoft account, since calendar synchronization relies on integrating with one of these two providers.
- The development team has continued access to free-tier or student-tier usage of the Google Calendar API and Microsoft Outlook Calendar API for the duration of the project.
- Students are comfortable using a standard web browser and don't require a native mobile app for the initial release.
- The project timeline follows a single semester, which shapes the scope and sequencing of the five increments described in Section 2.1.

Dependencies:

Athena depends on the following external components and services:

- Google Calendar API and Microsoft Outlook Calendar API, for pulling deadlines and events into the to-do tracker.
- PDF.js (or an equivalent open-source library), for rendering and annotating PDFs directly in the browser.
- Cloud storage service (e.g., AWS S3 or equivalent), for hosting uploaded study material.
- OAuth 2.0 authentication provider, for secure student login and calendar account linking.

Changes to any of these third-party services, including API rate limits, pricing changes, or deprecated endpoints, could require adjustments to the affected module's design or implementation.

3. External Interface Requirements

3.1 User Interfaces

Athena provides a clean, student-friendly web interface built around a persistent sidebar for navigating between the five core modules: to-do tracker, study timer, resource manager and PDF viewer, calendar sync, and study group collaboration.

Design Principles

The Athena interface needs to:

- Keep a consistent visual language and layout across all modules, so a student moving from the PDF viewer to the to-do tracker doesn't feel like they've entered a different application.
- Prioritize clarity and quick access over decorative elements, given that students often use Athena in short bursts between classes.
- Support standard responsive layouts, so the interface remains usable on both laptop and tablet screen sizes.

Standard Interface Components

The following elements would appear consistently wherever relevant:

- Persistent sidebar for module navigation
- Search bar for finding tasks, resources, or notes
- Notification indicator for upcoming deadlines and group activity
- Confirmation dialogs for destructive actions (e.g., deleting a task or resource)
- Loading indicators for PDF uploads and calendar sync operations

Feedback

Athena needs to give clear feedback for significant actions through success messages, error banners, and inline validation prompts, so a student always knows whether an action (like syncing a calendar or submitting a group note) succeeded or failed.

Detailed screen layouts and wireframes would be documented separately in a UI design specification as development progresses.

3.2 Hardware Interfaces

Athena doesn't interact with any dedicated or specialized hardware. It runs entirely as a browser-based application on standard student devices.

Supported client devices include:

- Laptops and desktop computers
- Tablets, for viewing dashboards and PDFs

Athena requires no direct hardware integration (such as scanners, sensors, or specialized peripherals) since all functionality is delivered through the browser and standard internet connectivity.

3.3 Software Interfaces

Athena connects to a small set of external software components to deliver its core features.

Component	Purpose
Google Calendar API	Pulls student deadlines and events into the to-do tracker
Microsoft Outlook Calendar API	Alternative calendar provider for the same sync feature
PDF.js (or equivalent library)	Renders and enables annotation of PDFs directly in-browser
PostgreSQL / MongoDB	Stores student accounts, tasks, resources, and session data
Cloud storage (e.g., AWS S3)	Hosts uploaded study material and PDF files

OAuth 2.0 provider	Handles secure student login and calendar account linking
--------------------	---

Communication with the calendar APIs happens over REST, using JSON as the data format. Detailed API request and response formats would be documented separately as development progresses.

3.4 Communications Interfaces

Athena relies on standard web communication protocols to connect students, the backend server, and external services.

- **HTTPS** secures all communication between the student's browser and Athena's backend.
- **WebSocket** connections support real-time updates for study group collaboration, such as new group notes or messages appearing without a page refresh.
- **REST over HTTPS** is used for communication with the Google Calendar and Outlook Calendar APIs.
- **SMTP** (or an equivalent email service) handles any account-related notifications, such as password resets.

All data exchanged between Athena and external services is encrypted in transit, and calendar sync occurs on a scheduled basis (e.g., every few minutes) rather than continuously, to stay within third-party API rate limits.

4. System Features

This section describes the major features provided by Athena. Each feature is organized into its description and priority, stimulus/response sequences, and detailed functional requirements.

4.1 Authentication and Account Management

4.1.1 Description and Priority

Authentication and Account Management provides students with secure access to Athena and protects personal academic information.

Priority: High

4.1.2 Stimulus/Response Sequences

Scenario 1 – Student Registration

Student Action

1. The student selects Create Account.
2. The student enters the required registration details.
3. The student submits the registration form.

Athena Response

1. Athena validates the entered information.
2. Athena checks whether the account information is acceptable.
3. Athena creates the account when the details are valid.
4. Athena displays a successful registration message.

Scenario 2 – Student Login

Student Action

1. The student enters registered login credentials.
2. The student selects Login.

Athena Response

1. Athena verifies the credentials.
2. Athena creates a secure session.

3. Athena redirects the student to the main dashboard.

Scenario 3 – Invalid Login

Student Action

1. The student enters incorrect credentials.
2. The student selects Login.

Athena Response

1. Athena rejects the login attempt.
2. Athena displays an appropriate error message.
3. The student remains on the login page.

4.1.3 Functional Requirements

REQ-AUTH-001: Athena would provide a registration facility through which students can create an account.

REQ-AUTH-002: The registration form would validate required information before an account is created.

REQ-AUTH-003: Athena would prevent registration when required information is missing or invalid.

REQ-AUTH-004: Students can log in using their registered credentials.

REQ-AUTH-005: Athena would display an informative message when incorrect credentials are entered.

REQ-AUTH-006: Athena would establish an authenticated session after successful login.

REQ-AUTH-007: Students can securely log out of Athena.

REQ-AUTH-008: Athena would restrict access to private academic information until authentication is completed.

4.2 To-Do Tracker

4.2.1 Description and Priority

The To-Do Tracker helps students organize academic activities by maintaining tasks, deadlines, priorities, and completion status.

Priority: High

4.2.2 Stimulus/Response Sequences

Scenario 1 – Creating a Task

Student Action

1. The student selects Add Task.
2. The student enters the task title, description, deadline, and priority.
3. The student saves the task.

Athena Response

1. Athena validates the task information.
2. Athena stores the task.
3. Athena displays the new task in the task list.

Scenario 2 – Completing a Task

Student Action

1. The student selects an active task.
2. The student marks the task as completed.

Athena Response

1. Athena changes the task status to completed.
2. Athena updates the task list.

4.2.3 Functional Requirements

REQ-TODO-001: Students can create tasks with a title and description.

REQ-TODO-002: Students can assign a deadline to a task.

REQ-TODO-003: Students can assign a priority to a task.

REQ-TODO-004: Athena would display pending and completed tasks separately or with clearly distinguishable status indicators.

REQ-TODO-005: Students can edit an existing task.

REQ-TODO-006: Students can mark a task as completed.

REQ-TODO-007: Students can delete tasks that are no longer required.

REQ-TODO-008: Athena would organize tasks according to attributes such as deadline, priority, or completion status.

REQ-TODO-009: Athena should display an appropriate message when invalid task information is entered.

4.3 Study Timer

4.3.1 Description and Priority

The Study Timer helps students conduct focused study sessions using a countdown mechanism and supports Pomodoro-style study periods.

Priority: High

4.3.2 Stimulus/Response Sequences

Scenario 1 – Starting a Study Session

Student Action

1. The student opens the Study Timer.
2. The student selects a study duration.

3. The student selects Start.

Athena Response

1. Athena begins the countdown.
2. Athena continuously displays the remaining study time.

Scenario 2 – Pausing or Resetting the Timer

Student Action

1. The student selects Pause or Reset.

Athena Response

1. Athena pauses the current countdown or resets it according to the selected action.
2. Athena updates the timer display.

Scenario 3 – Timer Completion

Athena Event

1. The countdown reaches zero.

Athena Response

1. Athena indicates that the study session has ended.
2. Athena records the completed session where session tracking is enabled.

4.3.3 Functional Requirements

REQ-TIMER-001: Athena would provide a countdown timer for study sessions.

REQ-TIMER-002: Students can start a study session.

REQ-TIMER-003: Students can pause an active study session.

REQ-TIMER-004: Students can reset the current timer.

REQ-TIMER-005: Athena would indicate when the configured study duration reaches zero.

REQ-TIMER-006: Athena can record completed study sessions.

REQ-TIMER-007: Each recorded study session would be associated with the student who initiated it.

REQ-TIMER-008: Athena should reject invalid timer durations.

4.4 Resource Management

4.4.1 Description and Priority

Resource Management provides students with a centralized location for storing, organizing, and accessing academic study materials.

Priority: High

4.4.2 Stimulus/Response Sequences

Scenario 1 – Uploading a Resource

Student Action

1. The student selects Upload Resource.
2. The student selects an academic file.
3. The student confirms the upload.

Athena Response

1. Athena validates the selected file.
2. Athena stores the resource.
3. Athena displays the resource in the student's collection.

Scenario 2 – Removing a Resource

Student Action

1. The student selects a stored resource.
2. The student selects Delete.
3. The student confirms the action.

Athena Response

1. Athena removes the selected resource.
2. Athena updates the resource list.

4.4.3 Functional Requirements

REQ-RES-001: Students can upload supported academic resources.

REQ-RES-002: Athena would validate uploaded files before storing them.

REQ-RES-003: Athena would associate each stored resource with its corresponding student.

REQ-RES-004: Students can view their stored resources.

REQ-RES-005: Students can organize resources using folders, categories, or other supported metadata.

REQ-RES-006: Students can rename supported resources.

REQ-RES-007: Students can delete resources they no longer require.

REQ-RES-008: Athena should display an informative error message when an unsupported or invalid file is selected.

4.5 PDF Viewer and Annotation

4.5.1 Description and Priority

The PDF Viewer allows students to open academic PDF documents within Athena and use supported tools for reading and annotation.

Priority: High

4.5.2 Stimulus/Response Sequences

Scenario 1 – Opening a PDF

Student Action

1. The student selects a PDF resource.

2. Athena opens the document.

Athena Response

1. Athena renders the PDF inside the application.
2. Athena provides document navigation controls.

Scenario 2 – Adding an Annotation

Student Action

1. The student opens a PDF.
2. The student selects an annotation tool.
3. The student adds the annotation.
4. The student saves the change.

Athena Response

1. Athena displays the annotation on the document.
2. Athena preserves the annotation for future access.

4.5.3 Functional Requirements

REQ-PDF-001: Athena would allow supported PDF files to be opened within the browser.

REQ-PDF-002: Students can navigate between pages of an opened PDF.

REQ-PDF-003: Students can zoom the displayed document.

REQ-PDF-004: Students can search for text within supported PDF documents.

REQ-PDF-005: Athena can provide supported annotation tools.

REQ-PDF-006: Students can add annotations to supported PDF documents.

REQ-PDF-007: Athena would preserve saved annotations associated with the corresponding document.

REQ-PDF-008: Athena should display an informative message when a PDF cannot be opened or processed.

4.6 Calendar Synchronization

4.6.1 Description and Priority

Calendar Synchronization allows students to connect Athena with supported external calendar services and access synchronized academic schedules.

Priority: Medium

The project description identifies Google Calendar and Outlook as the planned external calendar integrations.

4.6.2 Stimulus/Response Sequences

Scenario 1 – Connecting a Calendar

Student Action

1. The student selects the Connect Calendar.
2. The student selects a supported calendar provider.
3. The student grants the requested permission.

Athena Response

1. Athena establishes the calendar connection.
2. Athena retrieves permitted calendar information.
3. Athena displays synchronized events.

Scenario 2 – Synchronization Failure

Athena Event

1. The external calendar service becomes unavailable.

Athena Response

1. Athena informs the student that synchronization could not be completed.

2. Athena keeps unrelated Athena features available.
3. Athena retries synchronization when appropriate.

4.6.3 Functional Requirements

REQ-CAL-001: Athena would provide an option to connect a supported external calendar.

REQ-CAL-002: Students can select a supported calendar provider.

REQ-CAL-003: Athena would use the provider's authorization mechanism when connecting an external calendar.

REQ-CAL-004: Athena can retrieve calendar events permitted through the established connection.

REQ-CAL-005: Athena would display synchronized calendar events in an organized calendar view.

REQ-CAL-006: Students can disconnect a previously connected calendar.

REQ-CAL-007: Athena should display an informative message when synchronization cannot be completed.

REQ-CAL-008: Failure of an external calendar service would not prevent students from using unrelated Athena features.

4.7 Study Group Collaboration

4.7.1 Description and Priority

Study Group Collaboration allows students to work together by creating study groups, sharing academic resources, and managing access to shared content.

Priority: Medium

This feature is planned as the final increment because it involves shared folders, real-time or near-real-time updates, and permissions.

4.7.2 Stimulus/Response Sequences

Scenario 1 – Creating a Study Group

Student Action

1. The student selects Create Study Group.
2. The student enters the required group information.
3. The student confirms creation.

Athena Response

1. Athena creates the study group.
2. Athena associates the creator with the group.
3. Athena provides appropriate group-management permissions.

Scenario 2 – Sharing a Resource

Student Action

1. The student selects a resource.
2. The student chooses a study group.
3. The student confirms sharing.

Athena Response

1. Athena verifies the student's permissions.
2. Athena makes the resource available to permitted group members.
3. Athena updates the shared resource list.

4.7.3 Functional Requirements

REQ-GROUP-001: Students can create study groups.

REQ-GROUP-002: A study-group creator can provide basic information about the group.

REQ-GROUP-003: Students can join a study group through the supported invitation or joining mechanism.

REQ-GROUP-004: Athena would maintain membership information for each study group.

REQ-GROUP-005: Authorized students can share supported resources with a

study group.

REQ-GROUP-006: Athena would restrict shared content according to the permissions associated with the group.

REQ-GROUP-007: Permitted group members can access resources shared with their group.

REQ-GROUP-008: Athena can provide shared folders or equivalent organization mechanisms for group resources.

REQ-GROUP-009: Athena would reflect relevant changes in shared group content to permitted members.

REQ-GROUP-010: Students without the required permission should not be able to modify protected group resources.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The following requirements describe Athena's expected responsiveness and performance during normal operation.

REQ-PERF-001: Athena should load the main dashboard within an acceptable response time under normal network conditions.

REQ-PERF-002: Common operations such as creating, updating, completing, and deleting a task should provide feedback without unnecessary delay.

REQ-PERF-003: The To-Do Tracker should display updated task information immediately after a successful operation.

REQ-PERF-004: The Study Timer should maintain accurate countdown behavior throughout an active study session.

REQ-PERF-005: The PDF Viewer should begin displaying a supported PDF within a reasonable time depending on document size and network conditions.

REQ-PERF-006: Athena should remain responsive while students navigate between major features such as the To-Do Tracker, Study Timer, Resource Manager, PDF Viewer,

and Calendar.

REQ-PERF-007: Calendar synchronization should provide status feedback when communication with an external calendar service is delayed.

REQ-PERF-008: Athena should support multiple students using independent features concurrently without significant performance degradation under the expected project workload.

REQ-PERF-009: Temporary unavailability of an external calendar service should not unnecessarily slow down unrelated Athena features.

5.2 Safety Requirements

Athena does not directly control physical equipment or perform actions that can cause physical harm. Safety requirements therefore focus mainly on preventing accidental loss or misuse of academic information.

REQ-SAFE-001: Athena should request confirmation before permanently deleting important resources, tasks, or study-group content.

REQ-SAFE-002: The Resource Manager should avoid accidental deletion of stored academic files through a single unintended action.

REQ-SAFE-003: Athena should validate important inputs before processing them to reduce the possibility of invalid academic data being stored.

REQ-SAFE-004: The PDF Viewer should clearly indicate when a document cannot be loaded instead of displaying incomplete or misleading content.

REQ-SAFE-005: Athena should provide clear warnings when an operation could result in permanent loss of stored information.

REQ-SAFE-006: When an external service such as a calendar provider is unavailable, Athena should communicate the failure without altering unrelated stored information.

5.3 Security Requirements

Security requirements focus on protecting student accounts, academic resources, study-group information, and calendar connections.

Authentication

REQ-SEC-001: Athena would require authentication before providing access to private academic information.

REQ-SEC-002: Athena should validate login credentials before creating an authenticated session.

REQ-SEC-003: Students should be able to securely terminate an active session through the logout function.

Data Protection

REQ-SEC-004: Athena should protect stored academic resources from unauthorized access.

REQ-SEC-005: A student's private tasks, resources, study sessions, and other personal information should remain inaccessible to other students unless explicitly shared.

REQ-SEC-006: Study-group resources would only be available to people with the required group permissions.

File Security

REQ-SEC-007: Athena would validate uploaded files before storing or processing them.

REQ-SEC-008: Athena should restrict access to stored files according to their ownership or sharing permissions.

External Services

REQ-SEC-009: Athena would use the authorization mechanism provided by an external calendar service when establishing calendar access.

REQ-SEC-010: Athena should avoid exposing or unnecessarily storing external calendar credentials.

Session Security

REQ-SEC-011: Athena should invalidate the active session after a student logs out.

REQ-SEC-012: Athena should prevent protected pages or resources from being accessed through an expired or invalid session.

5.4 Software Quality Attributes

Usability

REQ-QUAL-001: Athena should provide a clear and consistent interface across all major features.

REQ-QUAL-002: Frequently performed activities such as adding a task, starting a timer, opening a resource, and joining a study group should require only a reasonable number of interaction steps.

REQ-QUAL-003: Error messages should be understandable to students and should identify the action required to recover whenever possible.

Reliability

REQ-QUAL-004: Athena should preserve stored tasks, resources, study sessions, and study-group information during normal operation.

REQ-QUAL-005: Failure of one feature should not unnecessarily prevent unrelated features from functioning.

REQ-QUAL-006: Athena should handle invalid inputs and unexpected application conditions without terminating unexpectedly.

Maintainability

REQ-QUAL-007: Athena should maintain a modular structure so that individual features can be modified without unnecessary changes to unrelated features.

REQ-QUAL-008: The codebase should follow consistent naming, organization, and documentation practices to simplify future maintenance.

Scalability

REQ-QUAL-009: Athena should allow additional study-management features to be introduced without requiring a complete redesign of the existing application.

REQ-QUAL-010: The architecture should support an increasing number of students, tasks, resources, and study groups within the expected project scope.

Testability

REQ-QUAL-011: Individual Athena features should be testable independently before final integration.

REQ-QUAL-012: Functional components should support unit testing, integration testing, and regression testing.

Portability

REQ-QUAL-013: Athena should operate on commonly used modern web browsers.

REQ-QUAL-014: The web interface should remain usable on commonly used desktop and laptop screen sizes and should support responsive layouts where applicable.

Interoperability

REQ-QUAL-015: Athena should communicate with supported external calendar services through their available integration interfaces.

REQ-QUAL-016: External-service failures should be isolated so that they do not

unnecessarily affect core Athena functionality.

5.5 Business Rules

The following rules define the operating principles for Athena.

BR-001: Only authenticated students can access private academic information stored in Athena.

BR-002: Each task created through the To-Do Tracker remains associated with the student who created it unless it is explicitly shared through a supported collaboration feature.

BR-003: Students can modify or delete their own personal tasks and resources.

BR-004: Access to shared study-group content is determined by the permissions associated with the corresponding study group.

BR-005: Only students with appropriate group permissions can modify protected shared resources.

BR-006: A student cannot access another student's private resources without an explicit sharing mechanism.

BR-007: Calendar information is accessed only after the student grants the required permission to Athena.

BR-008: Disconnecting an external calendar should stop further synchronization from that calendar.

BR-009: Deleted resources should no longer appear in the student's active resource collection.

BR-010: Completed tasks should remain distinguishable from pending tasks until the student removes or otherwise archives them.

BR-011: A study session recorded by the Study Timer should remain associated with the student who started that session.

BR-012: Study-group membership determines which shared content a student can view or modify.

BR-013: Athena should not expose private academic information through error messages, URLs, or other publicly accessible interfaces.

BR-014: External service failures should not result in unauthorized changes to stored Athena data.

6. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>